

ACP Summer School 2026 Hackathon

Airport Turnaround Scheduling — Keeping Melbourne Airport Moving

Optimisation Challenge

Changchun, China, 24 - 28 August 2026



It is midnight at Melbourne Airport (YMML). Over the next twenty-four hours, hundreds of aircraft will land, park, swap their crews, refuel, swallow a new load of passengers and bags, and push back from their gates. Every one of them needs a place to park and a small army of ground crews to turn it around. Get it right and the apron looks like a well-rehearsed dance. Get it wrong and you have angry passengers, missed connections, a queue of taxiing aircraft burning fuel, and a very expensive day.

You are the operations planner. Your job today is to turn a stack of flight data and an IATA service handbook into a plan that an airport could actually run. We have done the data plumbing for you, and you decide how to model and solve the problem.

What you will build

An optimisation solution for airport-operations problem: given the day's flight schedule, the apron layout, the ground-service crew rosters, and the inter-gate travel matrix, produce a plan that assigns every flight to a gate, schedules every ground-handling task, and staffs each task with the crew units it needs. Hiring too many crews is wasteful; hiring too few is infeasible. The airport needs a plan that is both feasible and cost-effective.

You could use **use any optimisation technologies**, including MiniZinc, OR-Tools, Gurobi, a hand-rolled SAT encoding, or a pure Python heuristic. The only contract is the JSON instance file you read and the JSON solution file you produce.

Good luck. The apron is yours.

1 Gate Assignment

Every aircraft that lands needs a parking spot, and a spot at Melbourne airport is not just an arbitrary piece of tarmac. Each gate has a fixed stand size, sits at a specific terminal, supports either passenger or freighter operations, and has hours during which it is actually open.

Table 1 gives an example with five real flights. A flight can have different carrier, size and operation type. The carrier can be **domestic** or **international**, the size can be **narrow** or **wide**, and the operation type can be **passenger** or **freighter**. These attributes determine which gates are feasible for that flight. To model the time, we use integer 1-minute slots counted from the start of the instance window. For example, `flight_arr = 31` means 00:31, `flight_arr = 754` means 12:34 if we start our horizon at 00:00, and so on.

Flight	Arrival	Departure	Carrier	Size	Type
JST917	00:31	01:16	domestic	narrow	passenger
UAE9314	12:34	14:49	international	wide	passenger
JST034	04:02	04:47	domestic	narrow	passenger
MXD177	04:16	05:01	domestic	narrow	passenger
CPA3128	14:15	16:30	domestic	wide	freighter

Table 1: Example flights and their attributes.

Table 2 gives an example with four gates to play with. Similarly, a gate can have different stand size, usage and supported type. A gate’s stand size can be **small** or **large**.

Gate	Stand Size	Usage	Supported Type	Open	Close
D02	small	domestic	passenger	00:00	24:00
D03	large	international	passenger	00:00	24:00
D04	large	international	passenger	00:00	24:00
H03	large	domestic	freighter	06:00	24:00

Table 2: Example gates and their attributes. All times are local wall-clock.

We need a feasible gate assignment that respect all of the following:

- **C1.** Every flight is parked at one gate, and the flight’s whole stay (from `flight_arr` until `flight_dep`) must fall inside that gate’s `[open, close]` window.
- **C2.** A wide-body flight needs a large stand. Narrow-body flights fit either size.
- **C3.** A passenger flight goes to a passenger gate, a freighter flight to a freighter gate.
- **C4.** An international flight must go to an international gate, and a domestic flight to a domestic gate. Customs takes a dim view of mixing the two.
- **C5.** To handle uncertainty, two flights assigned to the same gate must be separated by at least a **30-minute** inter-turn gap: if flight f_1 departs at `flight_dep1` and flight f_2 arrives at `flight_arr2` on the same gate, then `flight_arr2 - flight_dep1 ≥ 30` minutes.

Once you have a gate assignment satisfying C1–C5 you have a **Tier 1** solution. Feasibility is easy on a few flights, and the problem gets interesting when we need to handle the roster of hundreds of flights in our largest instance.

2 Crew Scheduling

The aircraft has just landed and rolled into its gate. The clock is running. Before it can leave again it must finish a sequence of ground-handling tasks. Passengers disembark, baggage is unloaded, fuel is pumped, the cabin is cleaned, catering trolleys are loaded, fresh baggage goes on, the next passengers board, the doors close, and a tractor pushes the aircraft back. Some tasks can happen in parallel. Others have to wait.

Each task needs exactly *one* team of ground staff, present at the gate for the task's duration. The kind of team a task needs is fixed by the task itself (e.g. a **FUEL** task always needs a **fuel_team**). You decide which individual team does each task and when each task starts. The objective is the *minimum total cost* that still turns every aircraft around on time: hire too few crew and the schedule is infeasible, hire too many and it is wasteful.

There are seven kinds of ground crew, each doing a fixed set of task kinds:

- **ramp_team**: aircraft arrival securing (ARRIVE_SECURE), passenger disembarkation (DIS-EMBARK), international documents (INTL_DOCS) and passenger boarding (BOARDING).
- **baggage_team**: baggage unloading (BAG_UNLOAD) and loading (BAG_LOAD), cargo unloading (CARGO_UNLOAD) and loading (CARGO_LOAD).
- **fuel_team**: aircraft refuelling (FUEL).
- **catering_team**: catering (CATERING).
- **cabin_clean_team**: cabin cleaning (CABIN_CLEAN).
- **water_waste_team**: water/waste (WATER_LAV).
- **pushback_team**: pushback (PUSHBACK) for passenger flights, departure release (DEP_RELEASE) for freighters.

We have a fixed number of individual teams of each kind, each with a fixed cost and a shift window. Their name are specified in the **LaborID** array, and their kind, cost, and shift window are specified in the **labor_kind**, **labor_cost**, **labor_shift_start**, and **labor_shift_end** arrays. There is an upper limit on how many teams of each kind you can hire, but whether you actually hire them is up to you.

Every flight's task list is fixed by its *service profile*, namely the combination of carrier, aircraft size, and operation type. There are three structurally distinct task graphs, one per profile family. They are drawn in Figures 1, 2, and 3. Each task node is colour-coded by the labour *kind* of crew it requires: **ramp_team** **baggage_team** **fuel_team** **catering_team** **cabin_clean_team**

water_waste_team **pushback_team**. The nodes are also labelled with the duration range in minutes, in the form *narrow* / *wide*. Each task has a duration that may vary slightly from flight to flight but remains within the specified range.

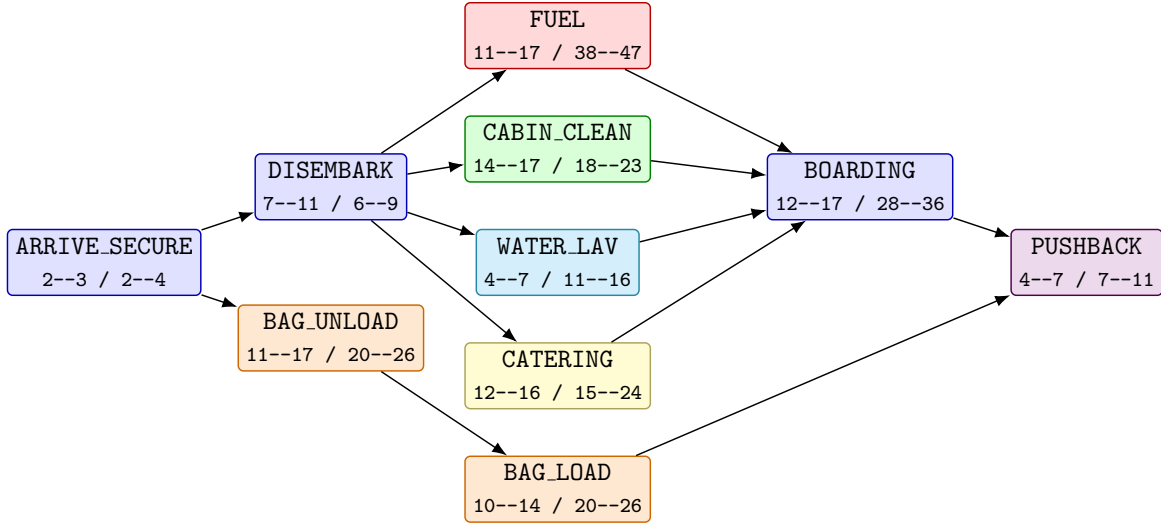


Figure 1: **Domestic passenger profile.** Durations in minutes.

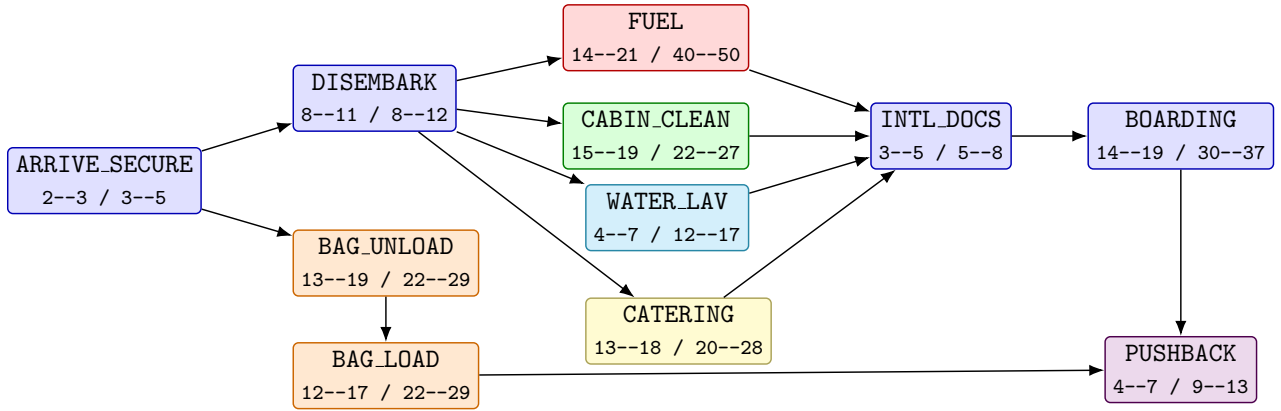


Figure 2: **International passenger profile.** Durations in minutes.

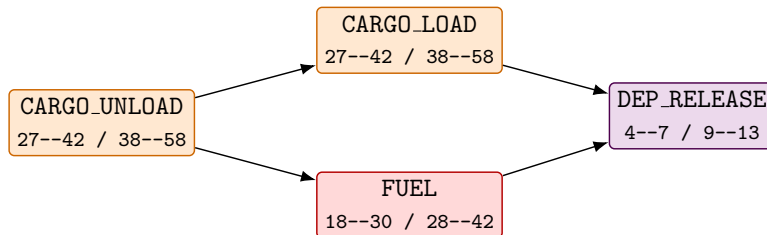


Figure 3: **Freighter profile.** Durations in minutes.

For every flight, we now additionally decide each task’s start time and which individual team performs it. On top of C1–C5, which still must hold, the start time and team assignment must satisfy:

- **C6.** The team you assign to a task is of the kind that task requires: the team’s `kind` must equal the required kind for that task kind.
- **C7.** Every task starts at or after the flight’s arrival and ends at or before its departure.
- **C8.** Whenever a flight has two tasks whose kinds are in a precedence relation (see Figures 1, 2, and 3), the predecessor must finish before the successor starts.
- **C9.** A team does one task at a time: no team is assigned to two tasks that overlap in time, whether on the same flight or on different flights.
- **C10.** Each team works only inside its own `shift` window from the start time in `labor_shift_start` to the end time in `labor_shift_end`: every task it performs must start and end within that window.

The objective is the *minimum total cost*: the sum of the costs of every team that appears *at least once* in your plan. Note that teams on different shifts can differ in cost.

This is a genuine *crew-scheduling* problem: you choose each task’s start time (within its window and precedence) and which shifts to staff, so that the concurrent same-kind tasks never exceed the teams you hire. Once C1–C10 hold, you have a **Tier 2** solution.

3 Bonus: Routing and Rest

The draft plan looks perfect on paper, but you notice two things almost immediately. First, a single fuel-truck unit finishes at gate B22 and may start at H03 one minute later, but the truck takes time to drive across the airfield. Second, a ramp team who has just spent 90 minutes back-to-back on three turnarounds is going to need a break before the next one.

Suppose every crew unit moves around the apron at the same speed, and the time to travel between any two gates is known. Now let's make each individual team's timeline physically real. Two further constraints now bind:

- **C11.** A team must have enough time to travel between the gates of two consecutive tasks assigned to it. If two tasks are assigned to the same gate, the travel time is zero of course.
- **C12.** A team cannot stay on the clock too long without a break. Walk a team's tasks in time order and track the start of its current work streak (its first task starts one). The streak carries across short gaps and across travel, and resets only when the team gets at least 15 minutes of break between two consecutive tasks. A team may not start a task more than 90 minutes after its current streak began. Note that travel does not count as rest, and the idle time is the gap between two tasks minus the travel needed between the two gates. For example, a team that has been on the clock 80 minutes and then spends 15 minutes travelling to the next gate has still had no break, and may not start a job immediately.

The instance ships a symmetric integer matrix `travel` indexed by gates, where `travel[g1][g2]` gives the minutes to move a unit from gate `g1` to gate `g2`. Same-gate transitions are zero. For any two consecutive tasks (f_1, t_1) and (f_2, t_2) assigned to the same unit with `task_end[f1, t1] ≤ task_start[f2, t2]`, the gap must cover the travel between their parking gates of f_1 and f_2 :

$$\text{task_start}[f_2, t_2] \geq \text{task_end}[f_1, t_1] + \text{travel}[\text{gate}[f_1], \text{gate}[f_2]].$$

. When the two gates are at opposite ends of the airfield, it can be deciding factor that forces you to hire more teams to finish all tasks.

Satisfying all constraints C1–C12 gives a **Tier 3** solution. Constraints C11 and C12 typically push the total hiring cost up compared with what C1–C9 alone would allow, because a unit that could previously chain two distant turnarounds now needs travel slack or rest, and you may need to hire an extra unit of that kind to absorb the slack.

4 Data Schema

The instances and submissions are both in JSON format. The following shows an example instance with five flights and four gates with a twenty-four-hour horizon.

Listing 1: Instance JSON

```
{
  "horizon": 1440,
  "FlightID": ["JST917", "UAE9314", "JST034", "MXD177", "CPA3128"],
  "GateID": ["D02", "D03", "D04", "H03"],
  "LaborID": ["ramp_team_01", "ramp_team_02", "baggage_team_01",
    /* ... one per team unit ... */],
  "max_tasks": 11,

  /* Data for flights */
  "flight_arr": [31, 754, 242, 256, 855],
  "flight_dep": [76, 889, 287, 301, 990],
  "flight_carrier": ["domestic", "international", "domestic",
    "domestic", "domestic"],
  "flight_size": ["narrow", "wide", "narrow", "narrow", "wide"],
  "flight_op_type": ["passenger", "passenger", "passenger",
    "passenger", "freighter"],
  "flight_n_tasks": [10, 11, 10, 10, 4],
  "flight_tasks": [
    [ {"kind": "ARRIVE_SECURE", "duration": 2},
      {"kind": "DISEMBARK", "duration": 7},
      /* ... flight_n_tasks[0] real tasks, then padded to max_tasks ... */
      {"kind": "PUSHBACK", "duration": 0} ],
    /* ... one padded row per flight ... */
  ],

  /* Data for gates */
  "gate_stand_size": ["small", "large", "large", "large"],
  "gate_usage": ["domestic", "international", "international", "domestic"],
  "gate_op_type": ["passenger", "passenger", "passenger", "freighter"],
  "gate_open": [0, 0, 0, 360],
  "gate_close": [1440, 1440, 1440, 1440],

  /* Data for labor units */
  "labor_kind": ["ramp_team", "ramp_team", "baggage_team", ...],
  "labor_cost": [220, 231, 219, ...],
  "labor_shift_start": [0, 104, 104, ...],
  "labor_shift_end": [209, 313, 313, ...],

  /* Travel time matrix (|GateID| x |GateID| = 4x4) */
  "travel": [[0, 2, 3, 9], [2, 0, 1, 10], [3, 1, 0, 11], [9, 10, 11, 0]]
}
```


The instance JSON has several top-level keys:

- **horizon** is the length of the instance window, in 1-minute slots.
- **FlightID**, **GateID** and **LaborID** are ordered arrays of identifiers used in arrays below. The order of these arrays is fixed and must be respected in your output.
- There are several attributes for flights:
 - **flight_arr/flight_dep** give the time window.
 - **flight_carrier/flight_size/flight_op_type** give the service profile.
 - **flight_n_tasks** give the number of real tasks
 - **flight_tasks** give the ordered task list, each entry **{kind, duration}**. It is a rectangular array padded to **max_tasks** columns, only the first **flight_n_tasks** entries of each row are real; the padding entries (duration 0) must be ignored.
- There are several arrays for the attributes for gates:
 - **gate_stand_size, gate_usage, gate_op_type**
 - each gate has an open window [**gate_open, gate_close**].
- There are several attributes for the crew units: **labor_kind**, hiring **labor_cost**, and the shift window [**labor_shift_start, labor_shift_end**] it may work in.
- **travel** is a $|\text{GateID}| \times |\text{GateID}|$ symmetric matrix with non-negative entries in 1-minute slots (corridor travel across the pier/apron layout).

Solution schema. A submission is a JSON object with four keys. Each task is done by exactly one team, so **task_labor[f][t]** is a single **LaborID**.

Listing 2: Example solution submission

```
{
  "gate":      ["D02", "D03", "D02", "D02", "H03"],
  "task_start": [[31, 33, 33, 40, 40, 40, 40, 44, 54, 66, 0],
                 /* ... one row per flight, padded to max_tasks ... */],
  "task_labor": [ ["ramp_team_01", "ramp_team_01",
                  "baggage_team_01", "fuel_team_01",
                  /* ... one LaborID per task, padded ... */],
                 /* ... one row per flight ... */],
  "cost": 4231
}
```

- **gate**: array sized $|\text{FlightID}|$, holding the chosen **GateID** for each flight.
- **task_start**: 2D array sized $|\text{FlightID}| \times \text{max_tasks}$ holding the start slot (1-minute) of each task. Padding entries beyond a flight's **flight_n_tasks** are ignored and may be set to zero.
- **task_labor**: 2D array sized $|\text{FlightID}| \times \text{max_tasks}$; each entry is the single **LaborID** you assign to that task. Padding entries may be any valid **LaborID**.
- **cost**: the sum of the costs of the distinct labour units that appear anywhere in the plan.

In the example, every task of the first flight starts no earlier than 31 and ends no later than 76, the precedence graph is respected, and each task is done by a team of the required kind. The grader recomputes the cost as the sum over the distinct **LaborIDs** used and checks it matches the reported cost.

5 Competition and Ranking

There are multiple instances in this competition. Each instance is scored independently, and your total score is the sum across instances. The leaderboard ranks participating teams by that total.

Points on a single instance are awarded across three tiers. Tier 1 is a flat award as soon as the gate-feasibility constraints hold. Tier 2 and Tier 3 add points that scale with how cheap your plan is compared with the current best on the leaderboard. Table 3 summarises the scoring scheme.

Tier	Constraints	Points formula	Max per instance
1	C1–C5	20	20
2	C1–C10	$50 \times \min(1, \text{best_tier2_cost}/\text{cost})$	50
3	C1–C12	$30 \times \min(1, \text{best_tier3_cost}/\text{cost})$	30

Table 3: Per-instance points scheme.

Suppose `best_tier2_cost` and `best_tier3_cost` are the best costs for Tier 2 and Tier 3 solutions seen so far on the leaderboard for this instance. If your submission satisfies C1–C10 and reports a cost of `cost`, you get 20 points for Tier 1 plus up to 50 points for Tier 2, scaled by how close your cost is to the best so far. If your solution further satisfies C11 and C12, you can earn up to 30 additional points for Tier 3, again scaled by the best Tier 3 cost so far.

Worked example. On some medium instance, the leaderboard currently shows a Tier-2 best of 220 and a Tier-3 best of 240. You submit a plan that satisfies every constraint and reports a cost of 245. If this cost matches the plan decisions and all of C1–C12 hold, your points on that instance are

$$20 + 50 \times \min\left(1, \frac{220}{245}\right) + 30 \times \min\left(1, \frac{240}{245}\right) \approx 20 + 44.9 + 29.4 = 94.3 \text{ points.}$$

The same 245-cost plan submitted with a Tier-3 constraint (either C11 or C12) violated would grade at Tier 2 only and score $20 + 44.9 = 64.9$ points on that instance. A pure Tier-1 submission with any cost scores a flat 20.

Submitting. You can submit as often as you like through the competition website before 16:00 August 28. The leaderboard keeps your best solution per instance. The leaderboard will be hidden half an hour before the competition deadline, and the final scoring will be done on the best submission you made.